

Software Requirements Specification for SleepSync: Sleeping Routine Management System

Version 1.0

Prepared by:

Ciddy S. Macapinlac

James Ernest T. Wolfe

Michael Anthony J. Yong

Cebu Institute of Technology - University

Date: May 10, 2025

Table of Contents

1. Introduction
 1. Purpose
 2. Scope
 3. Definitions, Acronyms, and Abbreviations
 4. References
 5. Overview
2. Overall Description
 1. Product Perspective
 2. Product Features
 3. User Classes and Characteristics
 4. Operating Environment
 5. Design and Implementation Constraints
 6. User Documentation
 7. Assumptions and Dependencies
3. System Features
 1. User Authentication
 2. Personalized Sleep Schedule Management
 3. Sleep Tracking and Analysis
 4. Smart Alarm
 5. Relaxation and Wind-Down Routines
 6. Sleep Goal Setting and Progress Tracking
 7. Personalized Sleep Tips
4. System Architecture
 1. Architecture Diagram
 2. Technology Stack
 3. Component Description
5. Data Design
 1. Database Schema
 2. API Design
 3. Data Flow
6. External Interface Requirements
 1. User Interfaces
 2. Hardware Interfaces
 3. Software Interfaces
 4. Communication Interfaces
7. Technical Specifications
 1. Security Requirements
 2. Performance Requirements
 3. Scalability Requirements
 4. Reliability Requirements
 5. Portability Requirements
8. Non-functional Requirements

1. Usability
 2. Availability
 3. Maintainability
 4. Compliance
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a detailed description of the requirements for the SleepSync application. It specifies the intended functionality, constraints, and system attributes of the application. This document serves as the foundation for the design, development, and testing phases of the SleepSync project.

1.2 Scope

SleepSync is a comprehensive sleep management and tracking system designed to help users improve their sleep quality and habits. The application will be accessible through web browsers and Android mobile devices. The system will provide users with features such as sleep tracking, personalized sleep schedules, smart alarms, relaxation routines, sleep goal setting, and personalized recommendations.

This SRS covers the functional and non-functional requirements of SleepSync, including the system architecture, user interfaces, hardware interfaces, and data design. It is intended for use by developers, testers, and project managers involved in the development of the SleepSync application.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer

SRS	Software Requirements Specification
UI	User Interface
UX	User Experience
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
ORM	Object-Relational Mapping
JSON	JavaScript Object Notation

1.4 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
2. Spring Boot Official Documentation: <https://spring.io/projects/spring-boot>
3. React Documentation: <https://reactjs.org/docs/getting-started.html>
4. Android Developer Documentation: <https://developer.android.com/docs>

1.5 Overview

The remainder of this document is organized as follows:

- Section 2 provides an overall description of the SleepSync system, including product perspective, features, and constraints.
- Section 3 details the system features, including specific requirements, user stories, and use cases.
- Section 4 describes the system architecture and technology stack.
- Section 5 presents the data design, including database schema and API design.
- Section 6 describes the external interfaces, including user interfaces and hardware interfaces.
- Section 7 outlines the technical specifications, including security, performance, and scalability requirements.

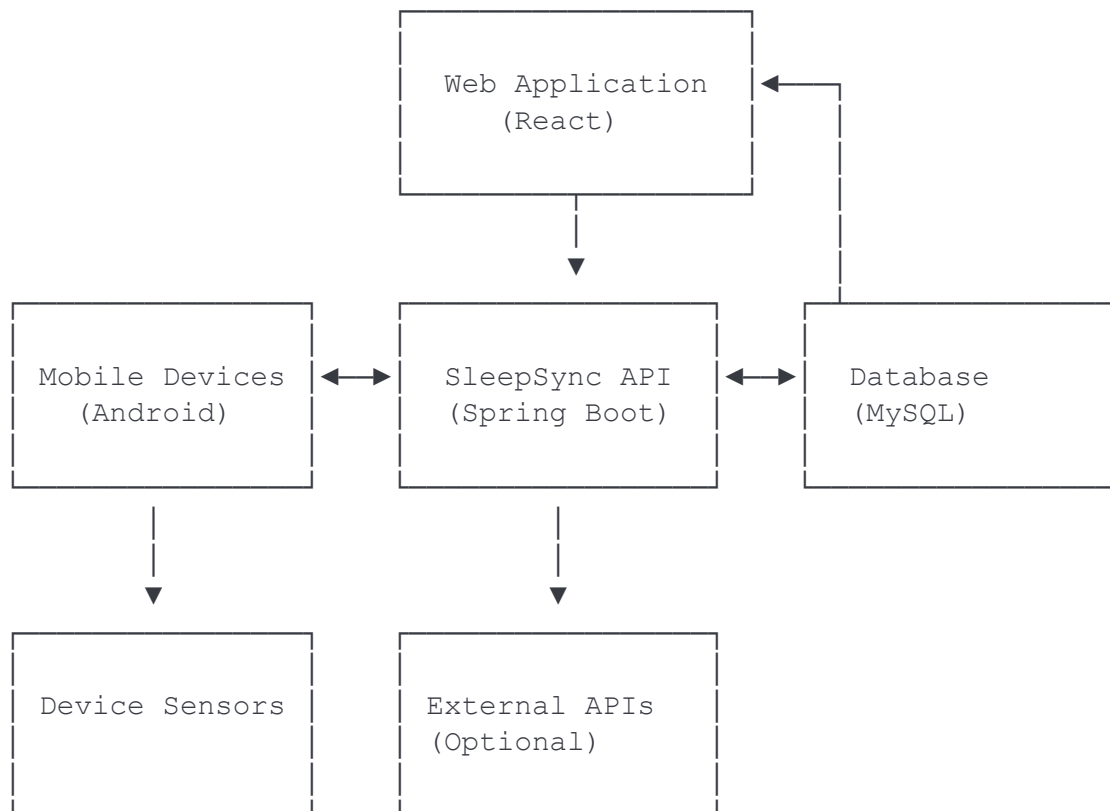
- Section 8 specifies the non-functional requirements.
- Section 9 contains appendices with supplementary information.

2. Overall Description

2.1 Product Perspective

SleepSync is a standalone cross-platform system designed to operate on web browsers and Android mobile devices. The application will integrate with device sensors to collect sleep data, such as movement and sound, to analyze sleep patterns. The system is composed of a React frontend for web platforms, an Android native application for mobile devices, and a Spring Boot backend with a MySQL database.

System Context Diagram



2.2 Product Features

SleepSync offers the following core features:

1. **User Authentication:** Secure registration and login functionality.
2. **Personalized Sleep Schedule Management:** Creation and management of personalized sleep schedules.

3. **Sleep Tracking and Analysis:** Recording and analyzing of sleep patterns and quality.
4. **Smart Alarm:** Alarm system that wakes users during light sleep phases.
5. **Relaxation and Wind-Down Routines:** Library of relaxation exercises and routines.
6. **Sleep Goal Setting and Progress Tracking:** Setting and monitoring sleep-related goals.
7. **Personalized Sleep Tips:** Recommendations based on user's sleep data and habits.

2.3 User Classes and Characteristics

SleepSync will cater to the following user classes:

1. **Regular Users:** Individuals seeking to improve their sleep quality and habits.
 - Characteristics: Varying levels of technical proficiency, interest in health monitoring.
 - Frequency of Use: Daily or several times a week.
 - Features Used: All core features.
2. **Health-Conscious Users:** Users with a strong focus on health optimization.
 - Characteristics: High interest in data analysis and trends, technically proficient.
 - Frequency of Use: Daily, with focus on analysis features.
 - Features Used: All features, with emphasis on sleep tracking, analysis, and goal setting.
3. **Users with Sleep Issues:** Individuals with specific sleep problems seeking improvement.
 - Characteristics: Focused on specific sleep-related concerns, varied technical proficiency.
 - Frequency of Use: Daily, with emphasis on solution-specific features.
 - Features Used: Sleep tracking, analysis, relaxation routines, and personalized tips.
4. **Administrators:** System administrators with access to backend systems.
 - Characteristics: High technical proficiency, responsible for system maintenance.
 - Frequency of Use: As needed for maintenance and support.
 - Features Used: Administrative interfaces and tools.

2.4 Operating Environment

The SleepSync system will operate in the following environments:

Web Application:

- **Browser Compatibility:** Chrome (v90+), Firefox (v88+), Safari (v14+), Edge (v90+)
- **Screen Resolutions:** Responsive design supporting desktop, tablet, and mobile viewports
- **Operating Systems:** Windows, macOS, Linux

Mobile Application:

- **Operating System:** Android 8.0 (API level 26) and higher

- **Device Types:** Smartphones and tablets
- **Required Sensors:** Accelerometer (for movement detection during sleep)
- **Optional Sensors:** Microphone (for sound detection), ambient light sensor

Backend Server:

- **Operating System:** Linux (recommended), Windows Server
- **Server Environment:** Java 17 or higher
- **Database:** MySQL 8.0 or higher

2.5 Design and Implementation Constraints

The development of SleepSync is subject to the following constraints:

1. **Technical Constraints:**
 - The web frontend must be developed using React.
 - The mobile application must be developed for the Android platform.
 - The backend must be developed using Spring Boot.
 - The database must be MySQL.
 - RESTful API architecture must be implemented for communication between components.
2. **Security Constraints:**
 - User authentication must be implemented using JWT.
 - All sensitive data must be encrypted in transit and at rest.
 - Compliance with relevant data protection regulations (e.g., GDPR).
3. **Hardware Constraints:**
 - The mobile application must operate efficiently on devices with limited resources.
 - Server infrastructure must support scalability for future growth.
4. **Time Constraints:**
 - The initial release must include all core features specified in this document.
 - Future enhancements will be addressed in subsequent releases.

2.6 User Documentation

The following user documentation will be provided with the SleepSync system:

1. **User Manuals:**
 - Web application user guide
 - Mobile application user guide

- Feature-specific tutorials
- 2. **Online Help:**
 - Context-sensitive help within the application
 - FAQ section
 - Troubleshooting guides
- 3. **Installation Guides:**
 - Mobile application installation guide
 - System requirements and configuration guide

2.7 Assumptions and Dependencies

The development of SleepSync is based on the following assumptions and dependencies:

1. **Assumptions:**
 - Users have basic knowledge of mobile applications and web interfaces.
 - Users have access to compatible devices for the web or mobile applications.
 - Users are willing to provide necessary permissions for sensor access on their devices.
2. **Dependencies:**
 - Availability of required hardware sensors on user devices.
 - Reliable internet connectivity for synchronization between devices.
 - Access to third-party libraries and frameworks for development.
 - Compatibility of the specified technology stack throughout the development lifecycle.

3. System Features

3.1 User Authentication

3.1.1 Description

The User Authentication feature provides secure registration, login, and account management functionality for SleepSync users. It ensures that user data is protected and accessible only to authorized individuals.

3.1.2 Functional Requirements

1. **User Registration**
 - The system shall allow users to register with an email address and password.
 - The system shall validate email addresses for correct format.
 - The system shall enforce password strength requirements (minimum 8 characters, at least one uppercase letter, one lowercase letter, one number, and one special character).
 - The system shall prevent duplicate registrations with the same email address.

- The system shall send a verification email to confirm the user's email address.
- 2. **User Login**
 - The system shall authenticate users using their email address and password.
 - The system shall issue a JWT token upon successful authentication.
 - The system shall implement protection against brute force attacks (e.g., account lockout after multiple failed attempts).
 - The system shall provide a "Remember Me" option for extended sessions.
 - The system shall provide a password recovery mechanism via email.
- 3. **Profile Management**
 - The system shall allow users to view and edit their profile information.
 - The system shall allow users to change their password.
 - The system shall allow users to delete their account.
 - The system shall allow users to set notification preferences.

3.1.3 User Stories

1. "As a new user, I want to create an account so that I can start using the SleepSync application."
2. "As a registered user, I want to log in securely so that I can access my sleep data."
3. "As a registered user, I want to reset my password if I forget it so that I can regain access to my account."
4. "As a registered user, I want to update my profile information so that my account details remain current."
5. "As a privacy-conscious user, I want to delete my account and all associated data so that I can protect my privacy."

3.1.4 Use Cases

Use Case: User Registration

- **Actor:** New User
- **Preconditions:** User has not previously registered
- **Main Flow:**
 - User navigates to the registration page
 - User enters email address, password, and confirms password
 - User submits the registration form
 - System validates the input
 - System creates a new user account
 - System sends a verification email
 - User verifies email address
 - System activates the user account
- **Alternative Flows:**
 - If email is invalid: System prompts user to enter a valid email
 - If password does not meet requirements: System displays password requirements
 - If email is already registered: System notifies user and suggests login

- **Postconditions:** User has a registered and activated account

Use Case: User Login

- **Actor:** Registered User
- **Preconditions:** User has a registered account
- **Main Flow:**
 - User navigates to the login page
 - User enters email and password
 - User submits the login form
 - System validates credentials
 - System generates and issues JWT token
 - System redirects user to the dashboard
- **Alternative Flows:**
 - If credentials are invalid: System displays error message
 - If account is locked: System notifies user and provides recovery options
- **Postconditions:** User is authenticated and has access to the application

3.2 Personalized Sleep Schedule Management

3.2.1 Description

The Personalized Sleep Schedule Management feature allows users to create and manage customized sleep schedules based on their preferences, goals, and lifestyle. The system provides recommendations for optimal sleep times and helps users maintain consistent sleep patterns.

3.2.2 Functional Requirements

1. Sleep Schedule Creation

- The system shall allow users to create personalized sleep schedules.
- The system shall collect user preferences regarding desired wake-up time, bedtime, and total sleep time.
- The system shall recommend optimal sleep and wake times based on user preferences and sleep science.
- The system shall allow users to set different schedules for weekdays and weekends.

2. Schedule Management

- The system shall allow users to view their current sleep schedule.
- The system shall allow users to modify existing sleep schedules.
- The system shall allow users to create multiple schedules for different scenarios (e.g., work days, vacations).
- The system shall provide a calendar view of the sleep schedule.

3. Reminders and Notifications

- The system shall send bedtime reminders based on the user's sleep schedule.
- The system shall send wake-up notifications based on the user's sleep schedule.

- The system shall allow users to customize reminder timing (e.g., 30 minutes before bedtime).
- The system shall provide gradual reminders leading up to bedtime (e.g., "Start winding down", "Prepare for bed").

3.2.3 User Stories

1. "As a user with inconsistent sleep patterns, I want to create a personalized sleep schedule so that I can establish a regular sleep routine."
2. "As a user with different weekday and weekend schedules, I want to set different sleep times for different days so that I can maintain appropriate sleep schedules."
3. "As a forgetful user, I want to receive bedtime reminders so that I can stay on track with my sleep schedule."
4. "As a user with changing work shifts, I want to create and switch between multiple sleep schedules so that I can adapt to my varying work hours."
5. "As a user trying to adjust my sleep schedule, I want the system to recommend gradual changes so that I can adapt more easily."

3.2.4 Use Cases

Use Case: Create Sleep Schedule

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has no existing sleep schedule
- **Main Flow:**
 - User navigates to the sleep schedule section
 - User enters preferred wake time and bedtime or desired sleep duration
 - System calculates optimal sleep schedule
 - System presents recommended schedule to user
 - User reviews and confirms the schedule
 - System saves the sleep schedule
- **Alternative Flows:**
 - If user preferences result in insufficient sleep: System suggests adjustments
 - If user has existing schedule: System asks if user wants to update or create new
- **Postconditions:** User has a personalized sleep schedule

Use Case: Manage Sleep Schedule Reminders

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has an existing sleep schedule
- **Main Flow:**
 - User navigates to reminder settings
 - User toggles bedtime and/or wake-up reminders
 - User sets reminder timing preferences
 - System saves reminder preferences

- System schedules notifications based on preferences
- **Alternative Flows:**
 - If notification permissions are not granted: System prompts user to enable notifications
- **Postconditions:** User has configured sleep schedule reminders

3.3 Sleep Tracking and Analysis

3.3.1 Description

The Sleep Tracking and Analysis feature enables users to record their sleep sessions and receive detailed analysis of their sleep patterns, quality, and trends over time. The system uses device sensors to detect sleep phases and provides insights on how to improve sleep quality.

3.3.2 Functional Requirements

1. **Sleep Recording**
 - The system shall allow users to manually start and stop sleep tracking sessions.
 - The system shall support automatic sleep detection using device sensors.
 - The system shall track sleep duration, estimated sleep phases, and sleep quality.
 - The system shall detect interruptions in sleep.
 - The system shall allow users to add notes to sleep sessions.
2. **Sleep Analysis**
 - The system shall analyze sleep data to identify patterns and trends.
 - The system shall provide visualizations of sleep metrics (e.g., sleep duration, sleep quality score, sleep phase distribution).
 - The system shall calculate sleep quality scores based on collected data.
 - The system shall identify potential sleep issues based on tracking data.
 - The system shall compare sleep metrics against recommended values.
3. **Sleep History**
 - The system shall maintain a history of sleep sessions.
 - The system shall allow users to view daily, weekly, and monthly sleep reports.
 - The system shall provide trend analysis over time periods (week, month, year).
 - The system shall allow users to export sleep data.

3.3.3 User Stories

1. "As a health-conscious user, I want to track my sleep patterns so that I can identify trends and areas for improvement."
2. "As a user with sleep concerns, I want to see a breakdown of my sleep phases so that I can understand my sleep quality."
3. "As a data-driven user, I want to view detailed sleep analytics over time so that I can measure improvements in my sleep habits."
4. "As a user with variable sleep quality, I want to add notes to my sleep sessions so that I can identify factors affecting my sleep."

5. "As a user seeking improvements, I want to see comparisons between my sleep metrics and recommended values so that I can set appropriate goals."

3.3.4 Use Cases

Use Case: Track Sleep Session

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has the application installed
- **Main Flow:**
 - User prepares for sleep
 - User opens the application and navigates to sleep tracking
 - User initiates sleep tracking
 - System monitors sleep using device sensors
 - User wakes up and stops tracking
 - System processes and analyzes sleep data
 - System presents sleep summary to user
- **Alternative Flows:**
 - If automatic tracking is enabled: System detects sleep automatically
 - If tracking is interrupted: System records partial session
- **Postconditions:** Sleep session is recorded and analyzed

Use Case: View Sleep Analysis Report

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has recorded sleep sessions
- **Main Flow:**
 - User navigates to sleep analysis section
 - User selects time period (day, week, month)
 - System generates sleep analysis report
 - System displays visualizations and insights
 - User reviews sleep patterns and trends
- **Alternative Flows:**
 - If no data exists for selected period: System prompts user to track sleep
- **Postconditions:** User has viewed sleep analysis report

3.4 Smart Alarm

3.4.1 Description

The Smart Alarm feature is designed to wake users during their lightest sleep phase within a designated time window, helping them feel more refreshed upon waking. The system monitors sleep patterns and activates the alarm at the optimal moment to minimize sleep inertia.

3.4.2 Functional Requirements

1. Alarm Configuration

- The system shall allow users to set smart alarm wake-up times.
- The system shall allow users to define a wake-up window (e.g., 30 minutes before the set time).
- The system shall allow users to select alarm sounds or music.
- The system shall support recurring alarms for specific days of the week.
- The system shall allow users to create multiple alarms.

2. Smart Wake-up

- The system shall monitor sleep phases during the wake-up window.
- The system shall identify the optimal wake-up time based on detected light sleep phases.
- The system shall gradually increase alarm volume for a gentle wake-up experience.
- The system shall provide a backup alarm at the final set time if no optimal time is detected.

3. Alarm Management

- The system shall allow users to snooze alarms.
- The system shall allow users to dismiss alarms.
- The system shall display upcoming alarms on the dashboard.
- The system shall allow users to temporarily disable alarms without deleting them.

3.4.3 User Stories

1. "As a user who struggles with morning grogginess, I want a smart alarm that wakes me during light sleep so that I feel more refreshed."
2. "As a user with a flexible morning schedule, I want to set a wake-up window so that the system can find the optimal time to wake me."
3. "As a user with different wake-up needs on different days, I want to create multiple smart alarms for different days of the week."
4. "As a user who prefers gentle awakening, I want my alarm to gradually increase in volume so that I can wake up peacefully."
5. "As a user who occasionally needs to adjust my schedule, I want to easily enable or disable alarms without deleting them."

3.4.4 Use Cases

Use Case: Set Smart Alarm

- **Actor:** Registered User
- **Preconditions:** User is authenticated
- **Main Flow:**
 - User navigates to the alarm section
 - User sets desired wake-up time
 - User defines wake-up window duration
 - User selects alarm sound and volume

- User sets recurrence pattern (if applicable)
- User saves the alarm
- System confirms alarm creation
- **Alternative Flows:**
 - If alarm conflicts with existing alarm: System notifies user
- **Postconditions:** Smart alarm is set and active

Use Case: Smart Wake-up Activation

- **Actor:** System
- **Preconditions:** User has active smart alarm and is sleeping
- **Main Flow:**
 - System begins monitoring sleep at start of wake-up window
 - System detects light sleep phase
 - System activates alarm with gradual volume increase
 - User responds to alarm
 - System logs wake-up time and sleep session details
- **Alternative Flows:**
 - If no light sleep detected in window: System activates alarm at final set time
 - If user doesn't respond: System continues alarm with increased volume
- **Postconditions:** User is awakened and alarm is recorded as completed

3.5 Relaxation and Wind-Down Routines

3.5.1 Description

The Relaxation and Wind-Down Routines feature provides users with guided activities and content designed to promote relaxation before bedtime. These routines help users transition from daily activities to sleep, improving sleep onset and quality.

3.5.2 Functional Requirements

1. Routine Library

- The system shall provide a library of relaxation and wind-down routines.
- The system shall categorize routines by type (e.g., meditation, breathing exercises, progressive muscle relaxation).
- The system shall offer routines of varying durations (5, 10, 15, 20 minutes).
- The system shall provide audio-guided routines with optional text descriptions.
- The system shall regularly update the routine library with new content.

2. Personalized Recommendations

- The system shall recommend routines based on user preferences and sleep data.
- The system shall adapt recommendations based on routine effectiveness for the user.
- The system shall suggest appropriate routines based on time available before bedtime.

- The system shall identify frequently used routines and provide easy access.

3. Routine Execution

- The system shall guide users through selected routines with audio and/or visual cues.
- The system shall track completion of routines.
- The system shall minimize screen brightness during routine execution.
- The system shall provide background sounds or music during routines.
- The system shall allow users to pause, resume, or exit routines.

3.5.3 User Stories

1. "As a user with racing thoughts at bedtime, I want guided meditation routines so that I can calm my mind before sleep."
2. "As a user with limited pre-bedtime time, I want to filter routines by duration so that I can find activities that fit my schedule."
3. "As a user trying to establish a bedtime routine, I want the system to recommend effective relaxation activities so that I can improve my sleep onset."
4. "As a user with varying stress levels, I want different types of relaxation exercises so that I can choose based on my current needs."
5. "As a user who benefits from certain routines, I want easy access to my favorite relaxation activities so that I can use them consistently."

3.5.4 Use Cases

Use Case: Browse and Select Routine

- **Actor:** Registered User
- **Preconditions:** User is authenticated
- **Main Flow:**
 - User navigates to relaxation routines section
 - User browses available routines
 - User applies filters (type, duration, etc.)
 - User selects a routine
 - System displays routine details
 - User chooses to start the routine
- **Alternative Flows:**
 - If user requests recommendations: System suggests appropriate routines
- **Postconditions:** User has selected a routine

Use Case: Execute Wind-Down Routine

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has selected a routine
- **Main Flow:**
 - System begins routine with introduction

- System guides user through routine steps with audio and visual cues
- System provides background sounds/music as appropriate
- System completes routine with conclusion
- System records routine completion
- System prompts user for feedback
- **Alternative Flows:**
 - If user pauses: System pauses and waits for resume
 - If user exits: System records partial completion
- **Postconditions:** User has completed relaxation routine

3.6 Sleep Goal Setting and Progress Tracking

3.6.1 Description

The Sleep Goal Setting and Progress Tracking feature enables users to establish specific sleep-related goals and monitor their progress over time. This feature helps users improve their sleep habits through measurable objectives and regular feedback.

3.6.2 Functional Requirements

1. Goal Setting

- The system shall allow users to set sleep-related goals (e.g., sleep duration, bedtime consistency, reduced wake-ups).
- The system shall provide goal templates for common sleep objectives.
- The system shall allow users to set target dates for goal achievement.
- The system shall enable users to create custom goals with personalized metrics.
- The system shall help users set realistic and achievable goals.

2. Progress Tracking

- The system shall track progress toward sleep goals using collected sleep data.
- The system shall display goal progress using visual indicators (e.g., progress bars, charts).
- The system shall provide daily, weekly, and monthly progress updates.
- The system shall identify trends in goal achievement.
- The system shall allow users to view historical goal performance.

3. Feedback and Motivation

- The system shall send notifications about goal progress.
- The system shall celebrate goal achievements with badges or rewards.
- The system shall provide recommendations when goals are not being met.
- The system shall adapt goals based on user performance and feedback.
- The system shall encourage consistency through streaks and milestones.

3.6.3 User Stories

1. "As a user trying to establish better sleep habits, I want to set specific sleep goals so that I have clear objectives to work toward."

2. "As a goal-oriented user, I want to track my progress toward sleep goals so that I can see my improvements over time."
3. "As a user who responds to positive reinforcement, I want to receive recognition for achieving sleep goals so that I stay motivated."
4. "As a user with changing priorities, I want to modify my sleep goals so that they remain relevant and achievable."
5. "As a user seeking guidance, I want recommendations for appropriate sleep goals so that I can focus on impactful improvements."

3.6.4 Use Cases

Use Case: Set Sleep Goal

- **Actor:** Registered User
- **Preconditions:** User is authenticated
- **Main Flow:**
 - User navigates to goal setting section
 - User browses available goal templates or selects custom goal
 - User defines goal parameters (target value, timeframe)
 - System validates goal feasibility
 - User confirms goal creation
 - System saves the goal and begins tracking
- **Alternative Flows:**
 - If goal parameters are unrealistic: System suggests adjustments
- **Postconditions:** Sleep goal is established and tracking begins

Use Case: Review Goal Progress

- **Actor:** Registered User
- **Preconditions:** User is authenticated and has active goals
- **Main Flow:**
 - User navigates to goal progress section
 - User selects goal to review
 - System displays progress data and visualizations
 - System shows comparison to previous periods
 - System provides insights and recommendations
 - User reviews information
- **Alternative Flows:**
 - If goal is newly created: System indicates insufficient data
- **Postconditions:** User has viewed goal progress information

3.7 Personalized Sleep Tips

3.7.1 Description

The Personalized Sleep Tips feature provides users with customized recommendations and advice to improve their sleep habits and quality. The system analyzes user sleep data and patterns to deliver relevant and actionable tips.

3.7.2 Functional Requirements

1. Tip Generation

- The system shall analyze user sleep data to identify areas for improvement.
- The system shall generate personalized sleep tips based on identified issues and patterns.
- The system shall provide tips across various categories (e.g., sleep environment, pre-sleep routine, nutrition).
- The system shall prioritize tips based on potential impact on sleep quality.
- The system shall avoid repeating tips too frequently.

2. Tip Delivery

- The system shall display personalized tips on the user dashboard.
- The system shall send periodic tip notifications.
- The system shall allow users to save tips for future reference.
- The system shall present tips in an engaging and easily understandable format.
- The system shall provide additional resources related to tips when available.

3. Tip Feedback

- The system shall allow users to provide feedback on tips (e.g., helpful, not helpful).
- The system shall adapt future tip recommendations based on user feedback.
- The system shall track which tips are implemented by users.
- The system shall evaluate the effectiveness of tips based on subsequent sleep data.

3.7.3 User Stories

1. "As a user with inconsistent sleep quality, I want personalized sleep tips so that I can identify specific actions to improve my sleep."
2. "As a novice to sleep science, I want easily understandable sleep recommendations so that I can implement them without needing specialized knowledge."
3. "As an engaged user, I want to provide feedback on sleep tips so that future recommendations are more relevant to me."
4. "As a busy user, I want sleep tips prioritized by impact so that I can focus on the most effective changes first."
5. "As a user implementing changes, I want to save useful tips so that I can refer to them later."

3.7.4 Use Cases

Use Case: Receive Personalized Sleep Tips

- **Actor:** Registered User

- **Preconditions:** User is authenticated and has tracked sleep data
- **Main Flow:**
 - User navigates to the sleep tips section
 - System analyzes user's sleep data and patterns
 - System generates personalized tips based on analysis
 - System presents prioritized tips to user
 - User reviews tips
- **Alternative Flows:**
 - If insufficient data exists: System provides general sleep tips
- **Postconditions:** User has received personalized sleep tips

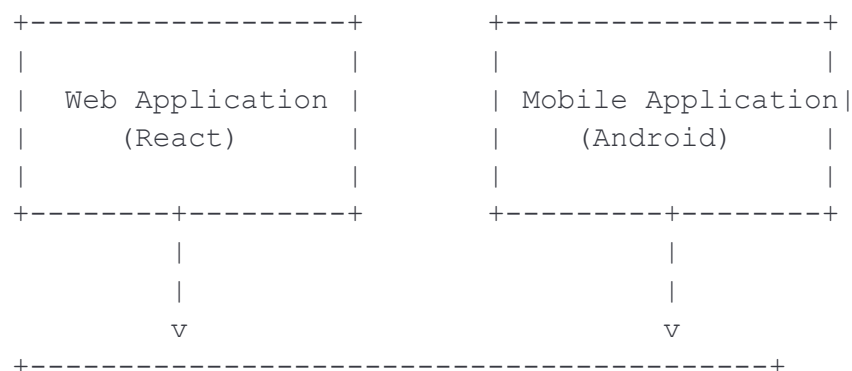
Use Case: Provide Feedback on Tips

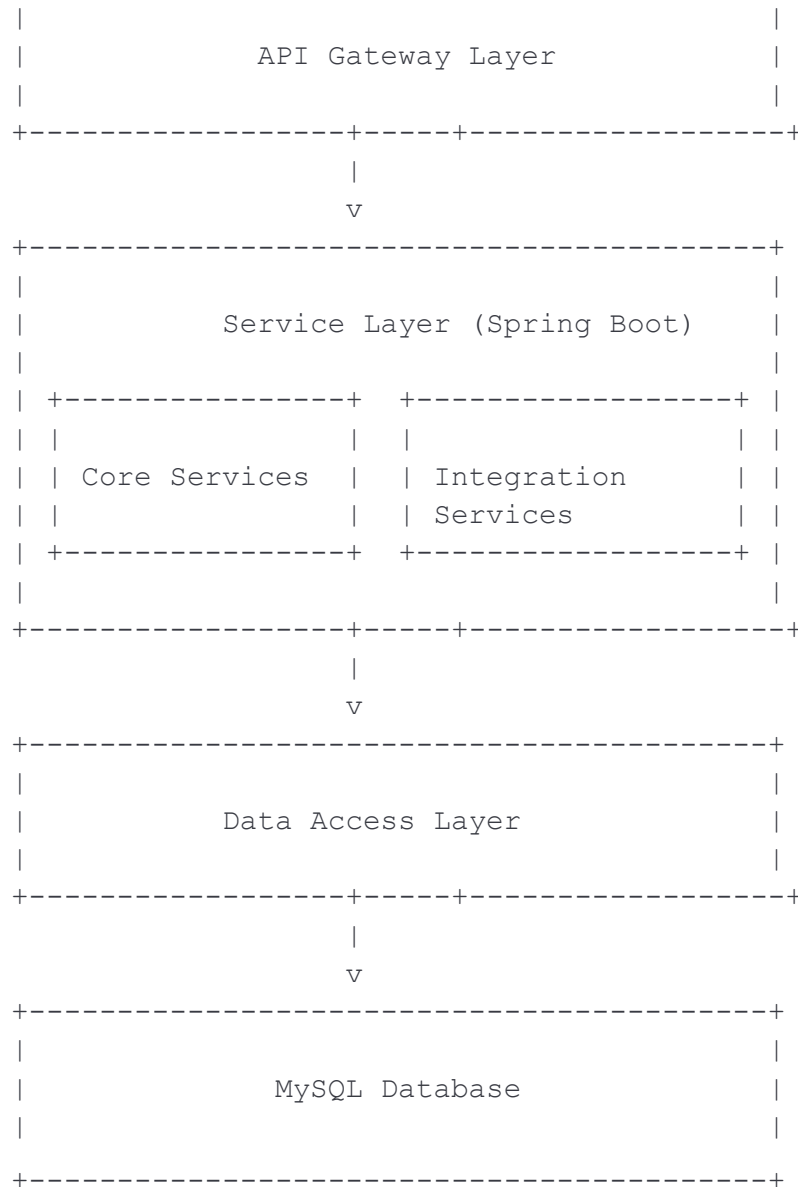
- **Actor:** Registered User
- **Preconditions:** User is authenticated and has received sleep tips
- **Main Flow:**
 - User reviews a sleep tip
 - User selects to provide feedback
 - User indicates whether the tip was helpful
 - User optionally provides additional comments
 - System records feedback
 - System adjusts future tip recommendations
- **Alternative Flows:**
 - None
- **Postconditions:** User feedback is recorded and influences future tips

4. System Architecture

4.1 Architecture Diagram

The SleepSync system follows a multi-tier architecture pattern with separate frontend applications for web and mobile platforms, a unified backend API, and a relational database. The system architecture is designed to be scalable, maintainable, and secure.





4.2 Technology Stack

The SleepSync system utilizes the following technology stack:

Frontend:

- **Web Application:**
 - React.js: Frontend library for building user interfaces
 - Redux: State management library
 - Material-UI: UI component library
 - Axios: HTTP client for API requests
 - Chart.js: Data visualization library

- Progressive Web App (PWA) capabilities
- **Mobile Application:**
 - Android (Java/Kotlin)
 - Android Jetpack Components
 - Room: Persistence library
 - Retrofit: HTTP client
 - MPAndroidChart: Data visualization
 - WorkManager: Background processing

Backend:

- **Application Server:**
 - Spring Boot: Application framework
 - Spring Security: Authentication and authorization
 - Spring Data JPA: Data access layer
 - Hibernate ORM: Object-relational mapping
 - JWT: Token-based authentication
 - Maven: Dependency management
- **Database:**
 - MySQL: Relational database management system

DevOps & Infrastructure:

- Docker: Containerization
- Jenkins or GitHub Actions: CI/CD
- AWS/Azure/GCP: Cloud hosting
- Logback: Logging framework
- Prometheus & Grafana: Monitoring

4.3 Component Description

4.3.1 Frontend Components

1. **Authentication Module:**
 - Handles user registration, login, and account management
 - Manages JWT tokens and user sessions
2. **Dashboard Module:**
 - Displays sleep metrics, insights, and summaries
 - Provides quick access to core features
3. **Sleep Tracking Module:**
 - Manages sleep recording sessions
 - Collects and processes sensor data
 - Displays sleep analysis results
4. **Sleep Schedule Module:**
 - Manages sleep schedules and reminders

- Provides schedule visualization
- 5. **Smart Alarm Module:**
 - Manages alarm settings and triggers
 - Implements smart wake-up algorithm
- 6. **Relaxation Module:**
 - Provides access to relaxation routines
 - Guides users through routine execution
- 7. **Goal Tracking Module:**
 - Manages sleep goals and progress
 - Provides goal visualizations and metrics
- 8. **Tips Module:**
 - Displays personalized sleep tips
 - Manages tip feedback and recommendations

4.3.2 Backend Components

1. **API Gateway:**
 - Routes requests to appropriate services
 - Handles authentication and authorization
 - Manages rate limiting and request validation
2. **User Service:**
 - Manages user accounts and profiles
 - Handles authentication and security
3. **Sleep Service:**
 - Processes and analyzes sleep data
 - Manages sleep tracking sessions
4. **Schedule Service:**
 - Manages sleep schedules
 - Handles reminders and notifications
5. **Alarm Service:**
 - Manages alarm configurations
 - Implements smart wake-up algorithms
6. **Relaxation Service:**
 - Manages relaxation content
 - Tracks routine usage and effectiveness
7. **Goal Service:**
 - Manages sleep goals
 - Tracks progress and achievements
8. **Recommendation Service:**
 - Generates personalized sleep tips
 - Analyzes user data for insights
9. **Notification Service:**
 - Manages and delivers push notifications
 - Handles scheduling of notifications
10. **Analytics Service:**

- Aggregates and analyzes user data
- Generates reports and insights

4.3.3 Data Components

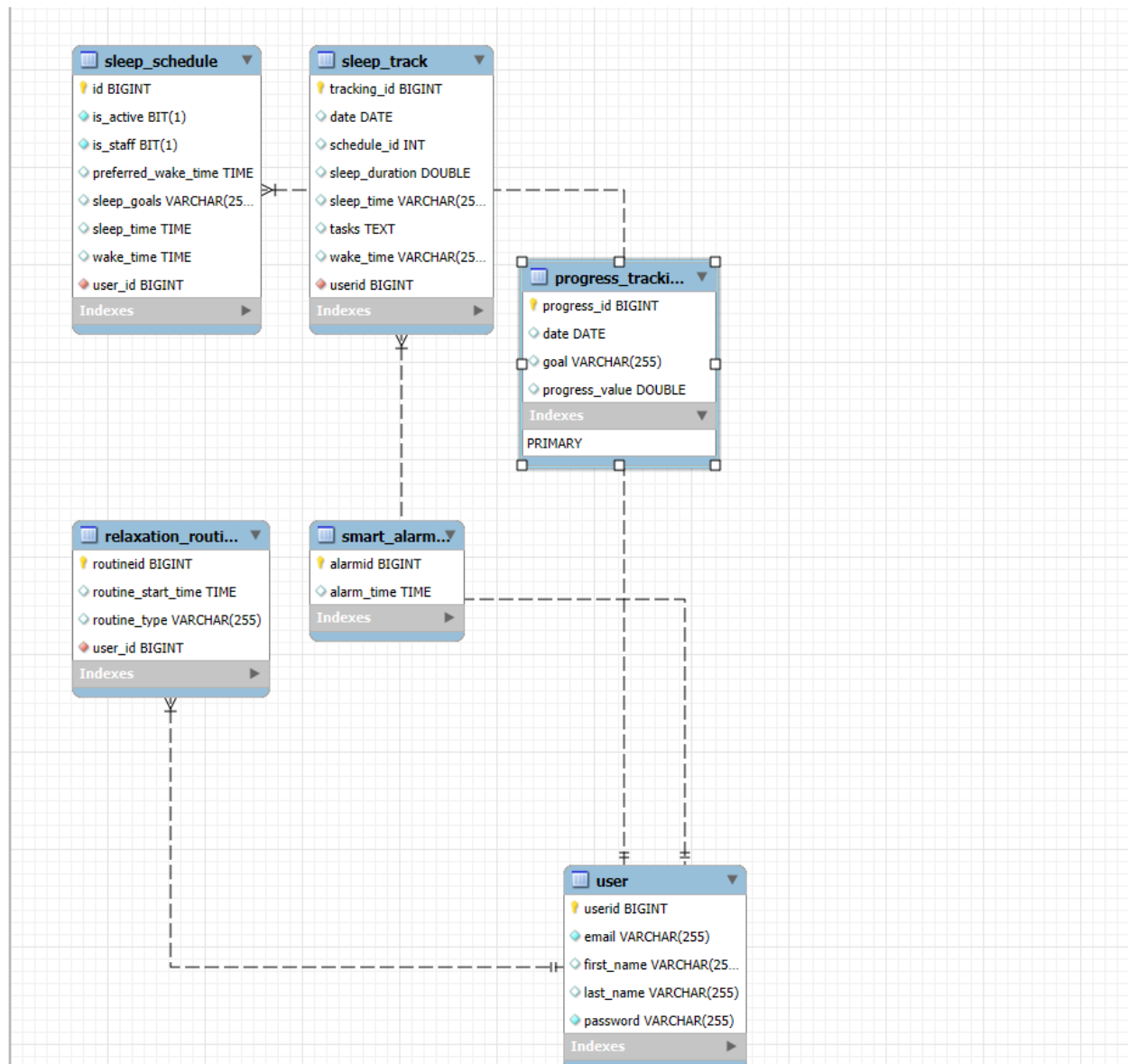
- 1. User Data Store:**
 - Stores user account information
 - Manages authentication data
- 2. Sleep Data Store:**
 - Stores sleep tracking records
 - Maintains sleep analysis results
- 3. Schedule Data Store:**
 - Stores sleep schedules
 - Manages reminder configurations
- 4. Content Data Store:**
 - Stores relaxation routines
 - Manages sleep tips and recommendations
- 5. Goal Data Store:**
 - Stores sleep goals
 - Tracks progress and achievements

5. Data Design

5.1 Database Schema

The SleepSync database schema is designed to efficiently store and retrieve data for all core features. The schema follows normalization principles and includes appropriate relationships between entities

Entity Relationship Diagram



Key Tables and Relationships:

1. **Users:** Stores user account information.
2. **SleepTracks:** Records sleep sessions with relationship to users.
3. **SleepSchedules:** Stores user sleep schedules.
4. **SmartAlarms:** Stores smart alarm configurations.
5. **RelaxRoutines:** Stores relaxation routine content.
6. **UserRoutines:** Tracks user completion of relaxation routines.
7. **GoalSettings:** Stores user sleep goals and progress.

5.2 API Design

The SleepSync API follows RESTful principles and is designed to provide secure and efficient access to the system's features. The API is organized into resources that correspond to the core features of the application.

Base URL:

`http://localhost:8080`

API Endpoints:

User Management:

- **POST /users** - Create a new user
 - Request Body: User registration information
 - Response: User object with authentication token
- **GET /users/{id}** - Get user by ID
 - Parameters: User ID
 - Response: User profile information
- **GET /users/login** - Login user
 - Request Body: Email and password
 - Response: Authentication token and user information
- **PUT /users/{id}** - Update user by ID
 - Parameters: User ID
 - Request Body: Updated user information
 - Response: Updated user object
- **DELETE /users/{id}** - Delete user by ID
 - Parameters: User ID
 - Response: Confirmation message

Sleep Schedule Management:

- **POST /api/sleep-schedules** - Create a new sleep schedule
 - Request Body: Schedule details
 - Response: Created schedule object
- **GET /api/sleep-schedules/{id}** - Get sleep schedule by ID
 - Parameters: Schedule ID
 - Response: Schedule details
- **GET /api/sleep-schedules** - List all sleep schedules for user
 - Parameters: User ID (query parameter)
 - Response: Array of schedule objects
- **PUT /api/sleep-schedules/{id}** - Update sleep schedule by ID
 - Parameters: Schedule ID
 - Request Body: Updated schedule details
 - Response: Updated schedule object

- **DELETE /api/sleep-schedules/{id}** - Delete sleep schedule by ID
 - Parameters: Schedule ID
 - Response: Confirmation message

Sleep Track Management:

- **POST /sleep-tracks/record_sleep_time/{id}** - Record sleep time for a user
 - Parameters: User ID
 - Request Body: Sleep tracking data
 - Response: Created sleep track object
- **GET /sleep-tracks/{id}** - Get all sleep tracks for a user
 - Parameters: User ID
 - Response: Array of sleep track objects
- **GET /sleep-tracks** - List all sleep tracks
 - Parameters: Pagination and filter options
 - Response: Array of sleep track objects
- **PUT /sleep-tracks/{id}** - Update sleep track by ID
 - Parameters: Track ID
 - Request Body: Updated tracking data
 - Response: Updated sleep track object
- **DELETE /sleep-tracks/{id}** - Delete sleep track by ID
 - Parameters: Track ID
 - Response: Confirmation message

Smart Alarm Management:

- **POST /api/smartalarms** - Create a new smart alarm
 - Request Body: Alarm configuration
 - Response: Created alarm object
- **GET /api/smartalarms/{id}** - Get smart alarm by ID
 - Parameters: Alarm ID
 - Response: Alarm details
- **GET /api/smartalarms** - List all smart alarms for user
 - Parameters: User ID (query parameter)
 - Response: Array of alarm objects
- **PUT /api/smartalarms/{id}** - Update smart alarm by ID
 - Parameters: Alarm ID
 - Request Body: Updated alarm configuration
 - Response: Updated alarm object
- **DELETE /api/smartalarms/{id}** - Delete smart alarm by ID
 - Parameters: Alarm ID
 - Response: Confirmation message

Relaxation Routine Management:

- **POST /relaxation-routines** - Create a new relaxation routine

- Request Body: Routine details
 - Response: Created routine object
- **GET /relaxation-routines/{id}** - Get relaxation routine by ID
 - Parameters: Routine ID
 - Response: Routine details
- **GET /relaxation-routines** - List all relaxation routines
 - Parameters: Category filter (query parameter)
 - Response: Array of routine objects
- **PUT /relaxation-routines/{id}** - Update relaxation routine by ID
 - Parameters: Routine ID
 - Request Body: Updated routine details
 - Response: Updated routine object
- **DELETE /relaxation-routines/{id}** - Delete relaxation routine by ID
 - Parameters: Routine ID
 - Response: Confirmation message

Progress Tracking:

- **POST /progress-tracking** - Create a new progress tracking record
 - Request Body: Goal progress data
 - Response: Created progress object
- **GET /progress-tracking/{id}** - Get progress tracking by ID
 - Parameters: Progress ID
 - Response: Progress details
- **GET /progress-tracking** - List all progress tracking records for user
 - Parameters: User ID (query parameter)
 - Response: Array of progress objects
- **PUT /progress-tracking/{id}** - Update progress tracking by ID
 - Parameters: Progress ID
 - Request Body: Updated progress data
 - Response: Updated progress object
- **DELETE /progress-tracking/{id}** - Delete progress tracking by ID
 - Parameters: Progress ID
 - Response: Confirmation message

5.3 Data Flow

The SleepSync system data flows are designed to ensure efficient processing and appropriate data handling across all components. The key data flows are described below:

User Registration and Authentication Flow:

1. User provides registration details through the UI
2. Frontend validates input and sends request to API
3. Backend validates data and creates user record

4. System generates authentication token
5. Token and user data returned to frontend
6. Frontend stores token for authenticated requests

Sleep Tracking Flow:

1. User initiates sleep tracking
2. Mobile sensors collect movement and sound data
3. Data processed locally and sent to backend
4. Backend analyzes data to identify sleep phases
5. Sleep metrics calculated and stored
6. Analysis results returned to user interface
7. UI displays sleep summary and insights

Smart Alarm Flow:

1. User configures smart alarm
2. Configuration stored in backend
3. During sleep tracking, system monitors sleep phases
4. When light sleep detected in wake window, alarm triggered
5. Alarm event recorded in history
6. User feedback on wake-up experience collected
7. Feedback used to improve future alarm timing

Sleep Tips Generation Flow:

1. System aggregates user sleep data
2. Analysis identifies trends and issues
3. Recommendation engine selects appropriate tips
4. Tips prioritized based on potential impact
5. Personalized tips delivered to user interface
6. User feedback collected on tip relevance
7. Feedback influences future tip selection

6. External Interface Requirements

6.1 User Interfaces

The SleepSync system will provide intuitive and responsive user interfaces for both web and mobile platforms. The interfaces will be designed with usability and accessibility as priorities.

Web Application UI:

1. **Dashboard:**
 - Sleep summary and key metrics

- Visual representations of sleep patterns
- Quick access to core features
- Upcoming alarms and reminders
- Personalized sleep tips
- 2. **Sleep Tracking:**
 - Sleep recording controls
 - Real-time tracking status
 - Summary of recent sleep sessions
 - Detailed sleep analytics
- 3. **Sleep Schedule:**
 - Visual calendar of sleep times
 - Schedule creation and editing interface
 - Reminder configuration
- 4. **Smart Alarm:**
 - Alarm creation and configuration
 - List of active alarms
 - Alarm history and effectiveness
- 5. **Relaxation Routines:**
 - Library of routines with filtering
 - Routine player with audio controls
 - Routine recommendation section
- 6. **Goals and Progress:**
 - Goal creation interface
 - Progress visualization
 - Achievement recognition

Mobile Application UI:

1. **Dashboard:**
 - Sleep summary card
 - Quick action buttons for core features
 - Upcoming alarm display
 - Sleep tips carousel
2. **Sleep Tracking:**
 - Prominent tracking button
 - Sleep session timer
 - Sensor status indicators
 - Recent sleep quality indicators
3. **Sleep Schedule:**
 - Weekly schedule view
 - Time picker for schedule adjustments
 - Reminder toggle controls
4. **Smart Alarm:**
 - Wake-up time setting with visual wake window
 - Sound selection interface

- Alarm list with toggle controls
- 5. **Relaxation Routines:**
 - Routine cards with preview
 - Full-screen routine player
 - Background audio controls
- 6. **Goals and Progress:**
 - Goal cards with progress indicators
 - Achievement badges
 - Trend visualizations

6.2 Hardware Interfaces

The SleepSync system will interface with various hardware components, primarily on mobile devices, to collect sleep-related data.

1. **Motion Sensors:**
 - Interface: Android Sensor API
 - Purpose: Detect movement during sleep
 - Requirements: Accelerometer access permission
2. **Microphone:**
 - Interface: Android Audio API
 - Purpose: Detect sounds during sleep (optional)
 - Requirements: Microphone access permission
3. **Ambient Light Sensor:**
 - Interface: Android Sensor API
 - Purpose: Detect light levels (optional)
 - Requirements: Light sensor access permission
4. **Vibration Motor:**
 - Interface: Android Vibration API
 - Purpose: Provide haptic feedback for alarms
 - Requirements: Vibration permission
5. **Speaker/Audio Output:**
 - Interface: Android Audio API
 - Purpose: Play alarm sounds and relaxation audio
 - Requirements: Audio output permission

6.3 Software Interfaces

The SleepSync system will interface with various software components and services.

1. **Operating System:**
 - Interface: Android API (for mobile)
 - Purpose: Access to device features and services
 - Requirements: Android 8.0 (API level 26) or higher
2. **Web Browser:**

- Interface: HTML5, CSS3, JavaScript
- Purpose: Render web application UI
- Requirements: Modern browser with JavaScript enabled
- 3. **Database Management System:**
 - Interface: JDBC/SQL
 - Purpose: Data storage and retrieval
 - Requirements: MySQL 8.0 or higher
- 4. **Push Notification Services:**
 - Interface: Firebase Cloud Messaging (FCM)
 - Purpose: Deliver notifications to devices
 - Requirements: FCM integration
- 5. **Background Processing:**
 - Interface: Android WorkManager (for mobile)
 - Purpose: Schedule and execute background tasks
 - Requirements: WorkManager API

6.4 Communication Interfaces

The SleepSync system will use the following communication interfaces:

1. **HTTP/HTTPS:**
 - Protocol: HTTP/1.1, HTTP/2
 - Purpose: Communication between clients and server
 - Requirements: SSL/TLS for secure connections
2. **WebSocket:**
 - Protocol: WebSocket Protocol (RFC 6455)
 - Purpose: Real-time updates for sleep tracking
 - Requirements: WebSocket support in clients and server
3. **Push Notifications:**
 - Protocol: FCM
 - Purpose: Deliver alerts and reminders
 - Requirements: Valid FCM configuration
4. **Database Connection:**
 - Protocol: JDBC
 - Purpose: Backend to database communication
 - Requirements: Connection pooling configuration

7. Technical Specifications

7.1 Security Requirements

The SleepSync system will implement comprehensive security measures to protect user data and ensure secure operations.

1. **Authentication and Authorization:**
 - JWT-based authentication
 - Role-based access control
 - Secure password storage using BCrypt hashing
 - Multi-factor authentication support (future enhancement)
 - Session management with token expiration
2. **Data Protection:**
 - Encryption of sensitive data at rest using AES-256
 - Transport layer security (TLS) for data in transit
 - Secure storage of API keys and credentials
 - Regular security audits and penetration testing
3. **Input Validation:**
 - Client-side and server-side validation
 - Protection against SQL injection
 - Protection against XSS attacks
 - CSRF protection
4. **Privacy:**
 - Compliance with relevant data protection regulations
 - Privacy-by-design implementation
 - Data minimization principles
 - User consent management
 - Clear data retention policies

7.2 Performance Requirements

The SleepSync system must meet the following performance requirements:

1. **Response Time:**
 - API response time: < 200ms for 95% of requests
 - Page load time: < 2 seconds for initial load
 - Real-time data updates: < 1 second delay
2. **Throughput:**
 - Support for 1,000 concurrent users
 - Handle 100 requests per second
 - Process 50 sleep tracking sessions simultaneously
3. **Resource Utilization:**
 - Mobile app memory usage: < 100MB
 - Mobile app CPU usage: < 10% during background tracking
 - Server CPU utilization: < 70% under peak load
 - Database connections: Properly managed pool
4. **Capacity:**
 - User database: Support for 1 million users
 - Sleep data: Handle 5 years of sleep data per user
 - File storage: Support for audio files up to 20MB each

5. **Mobile-Specific:**

- Battery impact: < 5% per hour during sleep tracking
- Storage usage: < 100MB for application installation
- Offline capability: Core functions available without connectivity

7.3 Scalability Requirements

The SleepSync system will be designed to scale effectively as the user base and data volume grow.

1. **Horizontal Scaling:**

- Stateless API design for easy replication
- Load balancing across multiple server instances
- Containerization for easy deployment of additional instances

2. **Database Scaling:**

- Connection pooling for efficient resource use
- Database sharding strategy for large user bases
- Read replicas for improved query performance

3. **Caching:**

- Implementation of multi-level caching
- Distributed cache for shared data
- Local caching for frequently accessed data

4. **Resource Management:**

- Auto-scaling based on load metrics
- Resource provisioning based on usage patterns
- Efficient database query optimization

5. **Batch Processing:**

- Asynchronous processing for non-critical operations
- Scheduled batch jobs for data aggregation and analysis
- Queue-based architecture for high-volume operations

7.4 Reliability Requirements

The SleepSync system must maintain high reliability to ensure user trust and data integrity.

1. **Availability:**

- System uptime: 99.9% (excluding scheduled maintenance)
- Scheduled maintenance windows with advance notice
- Redundant infrastructure for critical components

2. **Fault Tolerance:**

- Graceful degradation during partial failures
- Retry mechanisms for transient failures
- Circuit breaker patterns for dependent services

3. **Data Integrity:**

- Transaction management for critical operations

- Regular database backups
- Data validation at all system layers
- 4. **Error Handling:**
 - Comprehensive logging of system events
 - Detailed error tracking and reporting
 - User-friendly error messages
- 5. **Disaster Recovery:**
 - Regular backups with defined retention policy
 - Documented recovery procedures
 - Recovery time objective (RTO): < 4 hours
 - Recovery point objective (RPO): < 1 hour

7.5 Portability Requirements

The SleepSync system must be portable across different environments and platforms.

1. **Cross-Platform Support:**
 - Web application compatibility with major browsers
 - Mobile application support for various Android devices
 - Responsive design for different screen sizes
2. **Deployment Flexibility:**
 - Containerized deployment using Docker
 - Support for multiple cloud providers
 - On-premises deployment capability
3. **Configuration Management:**
 - Externalized configuration parameters
 - Environment-specific configuration files
 - Feature toggles for selective functionality
4. **Compatibility:**
 - Forward and backward compatibility for API versions
 - Graceful handling of version differences
 - Migration paths for data schema changes
5. **Internationalization:**
 - Support for multiple languages
 - Localization of user interfaces
 - Time zone handling for global users

8. Non-functional Requirements

8.1 Usability

The SleepSync system must provide an intuitive and accessible user experience to ensure adoption and continued use.

1. **User Interface:**
 - Clean and intuitive design
 - Consistent layout and navigation
 - Visual feedback for user actions
 - Appropriate typography and color contrast
2. **Accessibility:**
 - Compliance with WCAG 2.1 Level AA standards
 - Screen reader compatibility
 - Keyboard navigation support
 - Adjustable text size and contrast
3. **Learnability:**
 - Intuitive onboarding process
 - Contextual help and tooltips
 - Progressive disclosure of advanced features
 - Consistent interaction patterns
4. **Efficiency:**
 - Minimal steps required for common tasks
 - Quick access to frequently used features
 - Efficient data entry methods
 - Appropriate default values
5. **User Satisfaction:**
 - Aesthetically pleasing design
 - Engaging user experience
 - Appropriate feedback and confirmation
 - Error prevention and recovery

8.2 Availability

The SleepSync system must maintain high availability to ensure users can access their sleep data and features when needed.

1. **System Uptime:**
 - Target availability: 99.9% (excluding planned maintenance)
 - Planned maintenance during low-usage periods
 - Advance notification of scheduled downtime
2. **Offline Functionality:**
 - Core tracking features available without internet connection
 - Local data storage with synchronization upon reconnection
 - Clear indication of online/offline status
3. **Redundancy:**
 - Load balancing across multiple servers
 - Database replication and failover
 - Geographical distribution of resources
4. **Monitoring:**

- Real-time system health monitoring
- Automated alerts for performance issues
- Regular availability reporting

8.3 Maintainability

The SleepSync system must be designed for efficient maintenance and enhancement over time.

1. **Code Quality:**
 - Adherence to coding standards and best practices
 - Comprehensive documentation
 - Modular architecture with clear separation of concerns
 - Automated testing with high coverage
2. **Development Process:**
 - Version control with branching strategy
 - Continuous integration and deployment
 - Automated build and test processes
 - Code review requirements
3. **Debugging and Diagnostics:**
 - Comprehensive logging framework
 - Detailed error reporting
 - Performance monitoring tools
 - Diagnostic endpoints (secured appropriately)
4. **Documentation:**
 - Up-to-date architecture documentation
 - API documentation and examples
 - Development guides and onboarding materials
 - Operational procedures and runbooks

8.4 Compliance

The SleepSync system must comply with relevant regulations and standards to ensure legal operation and user trust.

1. **Data Protection:**
 - GDPR compliance for European users
 - CCPA compliance for California users
 - Appropriate data processing agreements
 - User rights implementation (access, deletion, etc.)
2. **Healthcare:**
 - Disclaimer about non-medical nature of the application
 - Clear separation from diagnostic or treatment claims
 - Appropriate terms of service regarding health data
3. **Accessibility:**
 - WCAG 2.1 Level AA compliance

- Accessibility statement and documentation
- Regular accessibility audits

4. Industry Standards:

- OAuth 2.0 for authentication where applicable
- REST API design principles
- Semantic versioning for APIs
- Standard security practices